策略邏輯 (Strategy Logic) (7%)

- **A. 資訊的使用 (Information Used)： (2%)** /E24146644 何家睿/

我的策略程式充分利用了平台提供的三個盤面資訊檔案，以建立完整的遊戲狀態並作為決策基底：

1. board.txt：用來建立 9x9 的棋盤陣列，解析每個座標點的棋子種類（1~3 為白方，4~6 為黑方）與空格。這是判斷移動合法性、棋子陣營以及追蹤雙方三種棋子（Tzaar, Tzarra, Totts）剩餘數量的最基礎資訊。
2. chessLayer.txt：同步建立 9x9 的高度陣列。在遊戲規則中，此資訊用來判定吃子時的「己方高度 >= 敵方高度」；在策略決定中，它更是決定「制空權」與「棋子強度加權」的關鍵指標。
3. stepHistory.txt：透過讀取此檔案的最後一行，程式能解析出目前為遊戲的「第幾回合（Round）」。這項資訊特別用於處理「第一回合白棋僅能執行一步（吃子）」的特殊規則限制

- **B. 策略的決定 (Decision Making)： (5%)**

我的策略決策核心基於 **MiniMax 演算法** 搭配 **Alpha-Beta 剪枝**，其中對於盤面做出評分的函數，主要有兩個：evaluate(詳細評斷 minimax 模擬之對局是好是壞）與 OrderMove(對所有合法步以做輕量評分，使「猜測較佳的下法」提早進入 minimax 模擬，提高剪枝效率)。依此二評估區塊來決定最佳動作。

- **致命傷與稀缺性(evaluate 中)：**
  1. 遊戲勝利條件為「讓對手絕種」。程式設計了嚴格的稀缺性懲罰，隨著數量的減少，逐漸扣分提高，強迫 AI 優先保護該棋種。若某棋種歸零則直接回傳極值（宣告勝/負）。
  2. 基礎各棋種數量統計加分，tzaar 加最多分。
  3. 為了使 AI 知道「好死不如賴活著」我把 depth 參數引入 evaluate，使滅絕的極值扣分會因對局模擬的深度有所不同。
- **制空權與射線威脅判定(evaluate 中):**
  1. 高度代表棋子的絕對防禦力(或進攻力)。程式會找出全盤最高的棋塔，

若我方獨佔最高塔，會給予 immortalBonus 加分。

2. 程式會朝六個方向發射射線偵測敵情，若被高度更高的敵棋瞄準會產生「絕對劣勢」扣分，反之則給予「攻擊優勢」加分。

- **行動力(evaluate 中)**：

  1. **承上射線掃描**的資訊運用:若此合法步為 ”E” 則沿六個射線方向計算雙方有幾顆棋子「具備合法的吃子目標」，每多一顆具備攻擊力的棋子就額外加分。這能引導 AI 避免陷入無法吃子的局面而落敗。

- **猜測排序(orderMove 中)**：為了提升 Alpha-Beta 剪枝效率，程式會在進入 minimax 遞迴前對所有的走法(getMoves 之 return)進行輕量的「猜測評分」。(只與場面上個棋種數量有關)包含：

  1. 預先計算盤面上所有棋子的數量。
  2. **吃子：**進攻邏輯 -> 滅族打擊、瀕危打擊、普通打擊(吃掉的敵人越高，加越多分)。
  3. **堆子:** 防守邏輯 -> 不可自殺覆蓋自己(曾經發生的慘痛經驗)、保護稀缺棋種、疊越高越好。

  當所有合法的步數經過 minimax 模擬以及最終的評估，會分別得到一個評分，此評分的高低就是我的決策依據，由 findbestMove 做決策及輸出的命令

策略程式架構 (Program Architecture) (10%)

- **使用什麼資料結構、如何切割策略程式模組 (Data structures used and how you modularize the strategy.)。(5%)**
  - 資料類別

1. **Move**：紀錄動作類型（E、S、P）與起終點座標(sx, sy, ex, ey)。
2. **UndoRecord**：紀錄被覆蓋格子的「原始棋種」與「原始高度」，專門用於狀態回溯。比 copy 全盤面省效能。
3. **MoveScore**：將 Move 與其猜測分數 guessScore 綁定，用於走法排序。
4. **Timer**：封裝 clock() 函式的計時器類別，提供精準的時間戳記計算。

   - 1, 2, 3 處理時以 vector 存取，此做法使程式易讀易寫。

- 模組切割

1. **I/O 模組(工具)**：readBoard、readLayer、writeOutput 等，負責與平台檔案對接。(輔助函數: transformMove 座標轉換成正確輸出格式)

2. **規則與狀態模組(工具)**：getMoves 負責生成合法步，applyUndoMove 與 undoMove 負責棋盤狀態的推進與復原。(輔助函數: isValid 、isOnBoard)

3. **評估模組(工具)**：evaluate、scanRaycast、getPenalty 與 fastCheckGameOver，專注於靜態盤面的分數計算與勝負快篩。(輔助函數:getPenalty)

4. **搜尋與決策模組(AI 本體)**：minimax、orderMoves 與 findBestMove，（輔助邏輯 alpha-beta 剪枝:套用於 minimax 跟 findBestMove）。

   - **比喻說明**: 我把整個 AI 本體想成一間公司
     - **大老闆**（findBestMove）： 負責建構決策樹、時間控管與輸出最終結果。
     - **情報員**（getMoves）： 屬於規則與狀態模組。負責在廣大的世界（棋盤）中，找出所有合法的步數。配合 applyUndoMove 與 undoMove（狀態的推進與復原）在各個平行宇宙間穿梭。
     - **地圖繪製者**（orderMoves）： 把情報員找回來的一大堆合法步，按照基本的邏輯，簡單整理成一張有優先順序的「地圖」，交給大老闆。
     - **智庫團隊**（minimax）： 大老闆依照地圖，叫智庫團隊去各個平行宇宙「走走看」，並在時間限制內回來匯報此路線的最終評分。
     - **智庫的評分表**（evaluate）： 智庫在平行宇宙中評斷盤面好壞的依據（包含 scanRaycast、getPenalty、fastCheckGameOver 等工具)。

   - **運作流程：** 大老闆（findBestMove）腦中有 Alpha-Beta 剪枝策略，知道目前最有利的路線能賺多少。並同步這個底線給智庫團隊(minimax)，智庫去平行宇宙模擬時，若發現這條路線注定比現有最佳路線還要差，就提早回歸不浪費時間（剪枝)。時間到時，大老闆依照地圖上每個路線的最終評分，交出最佳走法。

- **這些架構對於程式開發所帶來的影響（如易於除錯或效能優化）等(How the architecture helps development (debugging, optimization, etc.).)。(5%)**

我會重點提及本專案功能模塊中的幾個技術亮點。

效能佳：

1. **「狀態回溯」取代「一直拷貝」**：透過 applyUndoMove 與 UndoRecord 紀錄差異，搭配 undoMove 還原狀態。這避免了在深層 Minimax 遞迴中頻繁 copy 9x9 的二維陣列，將節點展開的速度提升。

2. **剪枝效率最大化與動態歷史繼承 ：** Alpha-Beta 剪枝的效能極度依賴「走法評估的順序」，若能最先探索到極佳的分支，就能及早提高 Alpha 值（我方分數下限），進而觸發大規模剪枝。本架構採用了「靜態預判」與「動態更新」雙管齊下的優化：

    A. **靜態猜測排序（orderMoves）**：在展開搜尋樹前，程式會先將合法步綁定 MoveScore 結構，利用輕便邏輯進行快速打分並降序排列

    B. **動態歷史更新（findBestMove 中的 Iterative Deepening）**：這是大幅縮小搜尋空間的核心關鍵。在迭代加深迴圈中，程式利用 finalBestM1 紀錄了「上一層深度（Depth N-1）」所找到的全局最佳步。當準備搜索下一層（Depth N）時，程式會主動遍歷 firstMoves 陣列，找到該歷史最佳步，並使用 std::swap 將其強制置換到陣列的最首位（firstMoves[0]）。

    C. **效能提升**：讓 Alpha-Beta 演算法第一步就踩中這個「歷史高分分支」，能瞬間將 alpha 閾值拉到極高。當演算法繼續評估剩餘的走法時，絕大多數的分支都會因為無法超越此一極高的 alpha 門檻，而迅速觸發 beta <= alpha 條件被直接截斷。這種動態繼承機制省去了大量重複的深層運算。

    D. **勝負快篩機制**：fastCheckGameOver 只掃描三種棋子的存活狀態，一旦確認雙方六種棋子都還在就立刻中斷掃描。這比完整的迴圈判定更省算力。(以防「必輸」或「必勝」情況還一直往下模擬)

易於維護與檢測：

- **動態時間防護網**：利用 Timer 類別搭配 C++ 的 try-catch 例外處理機制。一旦遞迴過程中發現超過時間限制（4.5 秒），便會立刻 throw runtime_error("TIMEOUT")，瞬間跳脫所有深層遞迴。這不僅徹底解決了 Bot

因超時被判違規的痛點，也讓**除錯時**利用這個「穿越功能」不怕陷入無窮迴圈，也可以在穿越時輸出一些想看的參數如 depth。(原本利用 GOTO 但是他只能在單一函數中)

- **功能解耦**：所有功能分模塊封裝，main 函數非常輕便，容易維護或迭代。例如:我可以只改 evaluate 去實驗增加機器人強度也不用擔心因此交不出作業。

補充: 開發路徑與版本迭代紀錄

本專案經歷了 10 個主要版本的迭代。在開發過程中，我秉持著「先能動，在變聰明」的想法，逐步更新優化效能與戰略盲點的問題。以下為核心進化歷程：

開發階段	實作、技術導入	解決之痛點	最終效益
V1 基礎	規則引擎 (getMoves)	無法合法對弈	確保模擬過程 100% 符合遊戲規則
V2 智力	演算法架構 (minimax, evaluate)	無依據下棋	可以依循策略下棋的架構、
V3 效能	狀態回溯 (UndoRecord)	記憶體頻繁拷貝導致慢速	廢除 memcpy，搜尋速度巨幅提升
V4 聰明	射線掃描、稀缺性懲罰	AI 缺乏危機意識與方向感	具備預判威脅與保護瀕危物種的戰略觀
V5 效能	Alpha-beta 剪枝，orderMove 預排	Minimax 跑不動	成功可以跑到深度 2
V6 深度	迭代加深、try-catch，動態剪枝	深度不足、剪枝效率低下。	加強剪枝成效，確保 4.5 秒內交卷。
V7~v10 實戰	物理防呆、晚死一點、evaluate 微調、minimax 跳出	跟同學對戰打輸時的情境	消除地平線效應，具備逆風防守能力，不會 minimax 莫名卡死